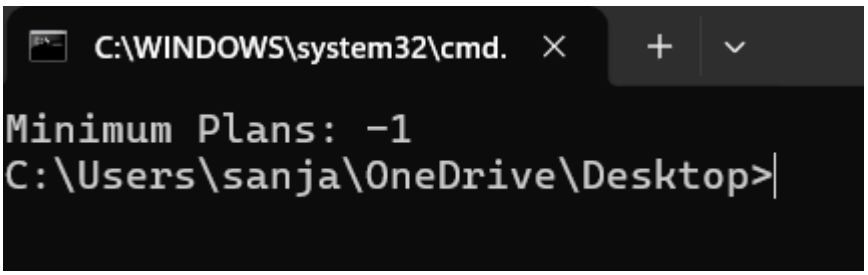
DAY 8:

1. Minimum Plans to Reach the Bandwidth

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int minPlans(vector<int>& plans, int target) {

   vector<int> dp(target+1, INT_MAX);
   dp[0] = 0;

   for(int p : plans){
      for(int i = target; i >= p; i--){
         if(dp[i-p] != INT_MAX)
            dp[i] = min(dp[i], dp[i-p] + 1);
      }
   }

   return dp[target] == INT_MAX ? -1 : dp[target];
}

int main(){

   vector<int> plans = {1,3,5};
   int target = 7;

   cout<<"Minimum Plans: "<<minPlans(plans,target);

   return 0;
}
```

OUTPUT:



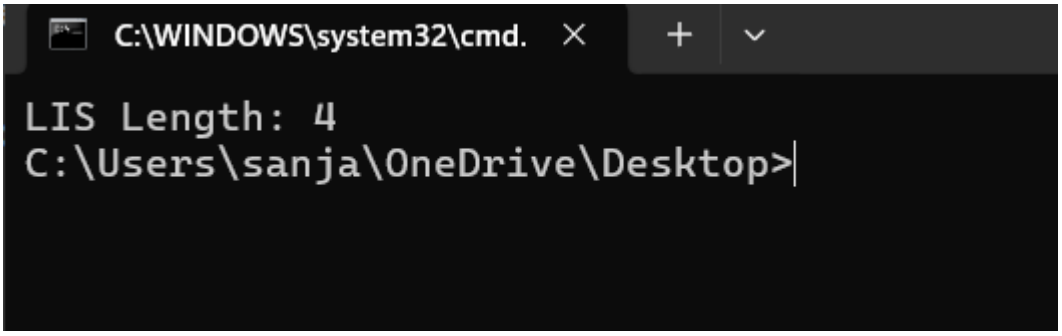
2. Longest Increasing Subsequence

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int lengthOfLIS(vector<int>& nums){

   vector<int> dp;
```

```
    for(int n : nums){

        auto it = lower_bound(dp.begin(), dp.end(), n);

        if(it == dp.end())
            dp.push_back(n);
        else
            *it = n;
    }

    return dp.size();
}

int main(){

    vector<int> nums = {10,9,2,5,3,7,101,18};

    cout<<"LIS Length: "<<lengthOfLIS(nums);

    return 0;
}
```

OUTPUT:



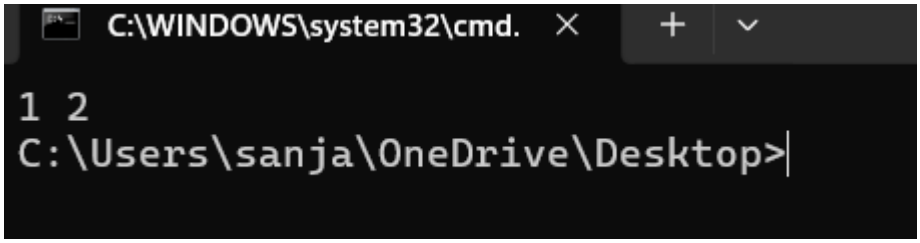
3. Top K Frequent Events

CODE:

```
#include <bits/stdc++.h>
using namespace std;

vector<int> topKFrequent(vector<int>& nums, int k){

    unordered_map<int,int> freq;

    for(int x : nums)
        freq[x]++;

    vector<pair<int,int>> arr;

    for(auto it : freq)
```

```
        arr.push_back({it.second,it.first});

    sort(arr.rbegin(), arr.rend());

    vector<int> res;

    for(int i=0;i<k;i++)
        res.push_back(arr[i].second);

    return res;
}

int main(){

    vector<int> nums = {1,1,1,2,2,3};
    int k = 2;

    vector<int> ans = topKFrequent(nums,k);

    for(int x : ans)
        cout<<x<<" ";

    return 0;
}
```

OUTPUT:



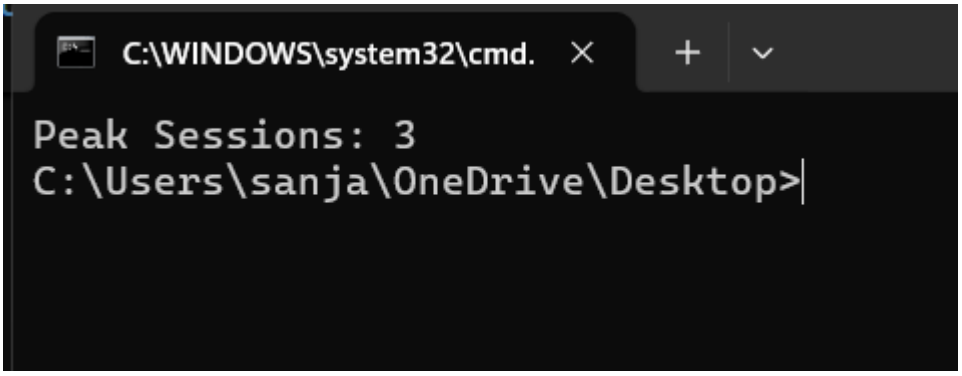
4. Peak Concurrent Sessions

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int peakSessions(vector<pair<int,int>> &sessions){

    vector<pair<int,int>> events;

    for(auto s : sessions){
        events.push_back({s.first,1});
        events.push_back({s.second,-1});
    }

    sort(events.begin(), events.end());
```

```
    int curr = 0, peak = 0;

    for(auto e : events){
        curr += e.second;
        peak = max(peak,curr);
    }

    return peak;
}

int main(){

    vector<pair<int,int>> sessions = {{1,5},{2,6},{4,7},{5,8}};

    cout<<"Peak Sessions: "<<peakSessions(sessions);

    return 0;
}
```

OUTPUT:



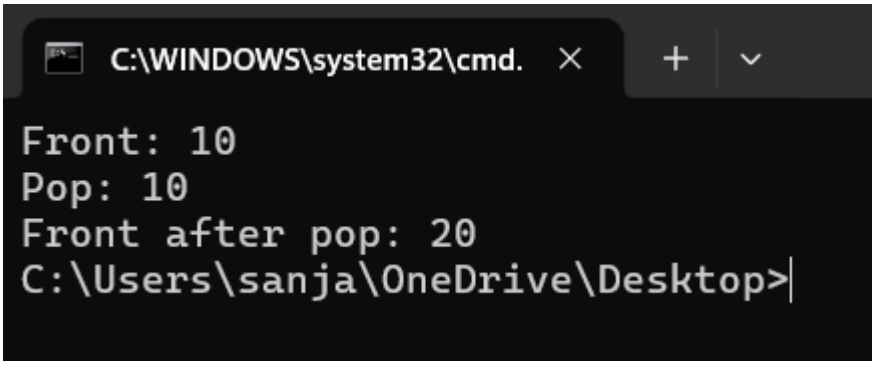
5. Queue using Two Stacks

CODE:

```
#include <bits/stdc++.h>
using namespace std;

class MyQueue{

    stack<int> s1,s2;

public:

    void push(int x){
        s1.push(x);
    }

    int pop(){

        if(s2.empty()){
```

```
        while(!s1.empty()){
            s2.push(s1.top());
            s1.pop();
        }
    }

    int val = s2.top();
    s2.pop();
    return val;
  }

  int peek(){

    if(s2.empty()){
        while(!s1.empty()){
            s2.push(s1.top());
            s1.pop();
        }
    }

    return s2.top();
  }
};

int main(){

  MyQueue q;

  q.push(10);
  q.push(20);
  q.push(30);

  cout<<"Front: "<<q.peek()<<endl;

  cout<<"Pop: "<<q.pop()<<endl;

  cout<<"Front after pop: "<<q.peek();

  return 0;
}
```

OUTPUT:



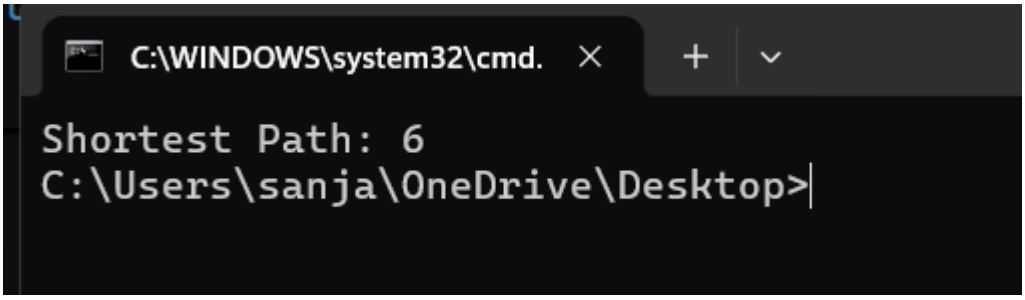
6. Shortest Path with Processing Delays (Dijkstra)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int shortestPath(int n, vector<vector<pair<int,int>>> &graph, int src, int dest){

    vector<int> dist(n, INT_MAX);

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    dist[src] = 0;

    pq.push({0,src});

    while(!pq.empty()){

        auto top = pq.top();
        pq.pop();

        int d = top.first;
        int u = top.second;

        for(auto edge : graph[u]){

            int v = edge.first;
            int w = edge.second;

            if(dist[v] > d + w){

                dist[v] = d + w;
                pq.push({dist[v],v});
            }
        }
    }

    return dist[dest];
```

```
}

int main(){

    int n = 5;

    vector<vector<pair<int,int>>> graph(n);

    graph[0].push_back({1,2});
    graph[0].push_back({2,4});
    graph[1].push_back({2,1});
    graph[1].push_back({3,7});
    graph[2].push_back({4,3});

    cout<<"Shortest Path: "<<shortestPath(n,graph,0,4);

    return 0;
}
```

OUTPUT:



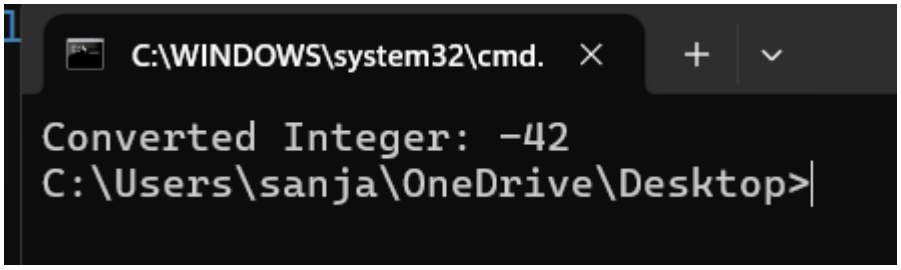
7. String to Integer (atoi)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int myAtoi(string s){

    int i = 0, sign = 1;
    long result = 0;

    while(i < s.size() && s[i] == ' ')
        i++;

    if(s[i] == '-' || s[i] == '+'){
        sign = (s[i] == '-') ? -1 : 1;
        i++;
    }

    while(i < s.size() && isdigit(s[i])){
```

```
        result = result*10 + (s[i]-'0');

        if(result*sign > INT_MAX) return INT_MAX;
        if(result*sign < INT_MIN) return INT_MIN;

        i++;
    }

    return result*sign;
}

int main(){

    string s = "   -42";

    cout<<"Converted Integer: "<<myAtoi(s);

    return 0;
}
```

OUTPUT:



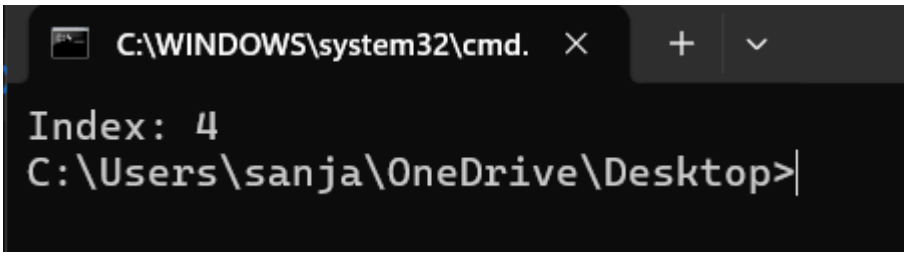
8. Search in Rotated Sorted Array

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int search(vector<int>& nums, int target){

    int l = 0, r = nums.size()-1;

    while(l <= r){

        int mid = (l+r)/2;

        if(nums[mid] == target)
            return mid;

        if(nums[l] <= nums[mid]){

            if(nums[l] <= target && target < nums[mid])
                r = mid-1;
```

```
            else
                l = mid+1;
        }

        else{

            if(nums[mid] < target && target <= nums[r])
                l = mid+1;
            else
                r = mid-1;
        }
    }

    return -1;
}

int main(){

    vector<int> nums = {4,5,6,7,0,1,2};
    int target = 0;

    cout<<"Index: "<<search(nums,target);

    return 0;
}
```

OUTPUT:



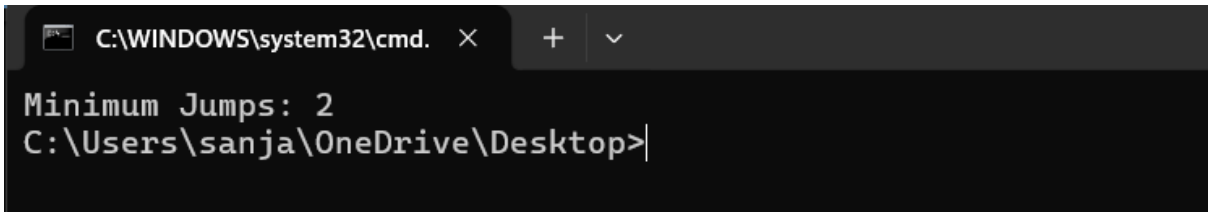
9. Jump Game II

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int jump(vector<int>& nums){

    int jumps = 0, currEnd = 0, farthest = 0;

    for(int i=0;i<nums.size()-1;i++){

        farthest = max(farthest,i+nums[i]);

        if(i == currEnd){
```

```
            jumps++;
            currEnd = farthest;
        }
    }

    return jumps;
}

int main(){

    vector<int> nums = {2,3,1,1,4};

    cout<<"Minimum Jumps: "<<jump(nums);

    return 0;
}
```

OUTPUT:



10. Minimum Spanning Tree (Prim's Algorithm)

CODE:

```
#include <bits/stdc++.h>
using namespace std;

int mst(int n, vector<vector<pair<int,int>>> &graph){

    vector<int> key(n,INT_MAX);
    vector<bool> inMST(n,false);

    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

    key[0] = 0;
    pq.push({0,0});

    int cost = 0;

    while(!pq.empty()){

        int u = pq.top().second;
        pq.pop();

        if(inMST[u]) continue;
```

```
        inMST[u] = true;
        cost += key[u];

        for(auto edge : graph[u]){

            int v = edge.first;
            int w = edge.second;

            if(!inMST[v] && w < key[v]){
                key[v] = w;
                pq.push({key[v],v});
            }
        }
    }

    return cost;
}

int main(){

    int n = 4;

    vector<vector<pair<int,int>>> graph(n);

    graph[0].push_back({1,10});
    graph[0].push_back({2,6});
    graph[0].push_back({3,5});
    graph[1].push_back({3,15});
    graph[2].push_back({3,4});

    cout<<"MST Cost: "<<mst(n,graph);

    return 0;
}
```

OUTPUT: